EncryptEdge Labs

Junior Cloud Security Engineer Internship Programme

Internship Task Report

Intern Full Name	[redacted]
Intern Email	[redacted]
Cohort	Cohort 7, 2026
Task ID	CSE-AWS-L-027-04
Task Title	AWS STS Assume-Role Abuse Scenarios and Trust Policy Hardening
Skill Domain	AWS Identity and Access Management (IAM) / Security Token Service (STS)
Difficulty Level	Advanced
CPE Credits	4 CPE Credits
Guiding Framework(s)	MITRE ATT&CK; NIST SP 800-53; ISO 27001; CIS Controls; OWASP Cloud Top 10; NICE Framework
Submission Date	31 Aug 2026

Copyright © 2026 EncryptEdge Labs Limited | Company No. 15830711 | United Kingdom

All activities in this report were performed within authorised lab environments as defined by the task brief.

1.0 Task Overview

Task ID	CSE-AWS-L-027-04
Task Title	AWS STS Assume-Role Abuse Scenarios and Trust Policy Hardening
Skill Domain	AWS Identity and Access Management (IAM) / Security Token Service (STS)
Difficulty	Advanced
Duration	14 Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	AWS Free Tier sandbox account (isolated, non-production) used with CloudGoat; browser-hosted PwnedLabs and Cybr lab environments
AWS Region(s) Used	us-east-1
Primary Tools	AWS CLI v2, IAM Policy Simulator, CloudGoat, jq, Visual Studio Code, Claude (policy review support)
Target Environment	CloudGoat scenario: iam_privesc_by_rollback; PwnedLabs — Assume Privileged Role with External ID; Cybr IAM Challenge Labs
MITRE ATT&CK IDs	T1078 (Valid Accounts), T1550.001 (Use of Web Session Cookie), T1098 (Account Manipulation)
Guiding Frameworks	NIST SP 800-53 AC-2/AC-6, CIS Control 6, OWASP Cloud Top 10 (IAM-01), AWS Well-Architected Framework — Security Pillar
Regulatory Alignment	ISO 27001 A.9.2.3 (Management of privileged access rights); NICE PR-CDA-001 (Cloud Security Analyst); SFIA Level 4 (Information Security)

1.1 Introduction

This task examines how AWS Security Token Service (STS) AssumeRole operations, when governed by loosely written trust policies, can be chained together to move from a low-privilege identity to full administrative access. Understanding this attack surface matters for a cloud security engineer because temporary-credential misuse is a leading root cause of real-world cloud breaches, and it is often invisible to teams that only review static IAM policies rather than the trust relationships between roles. The task targets the specific misconfiguration category of overly permissive AssumeRolePolicyDocument statements — missing sts:ExternalId, source-IP, and MFA conditions — that enable both cross-account and same-account privilege escalation.

1.2 Objectives & Learning Outcomes

- Analyze STS AssumeRole Mechanisms — Understand how temporary credentials are issued and used in AWS STS. Met by executing and inspecting assume-role calls end-to-end in Phase A and Phase C, including reviewing returned AccessKeyId/SecretAccessKey/SessionToken structures.
- Identify Cross-Account Trust Risks — Detect misconfigurations in IAM trust policies that allow unintended access. Met in Phase C by extracting and analysing the AssumeRolePolicyDocument of both lab roles and identifying the missing conditions.
- Simulate Role Chaining Attacks — Demonstrate how attackers can pivot between roles using chained assumptions. Met by completing a two-hop AssumeRole chain in Phase C that escalated from a lab user with no attached policies to a role carrying AdministratorAccess.
- Apply Trust Policy Hardening — Implement conditions such as aws:SourceArn, aws:SourceIp, and sts:ExternalId to restrict role assumption. Met in Phase D by rewriting the entry-point trust policy with ExternalId, source-IP, MFA, and session-duration conditions and confirming the original attack path was blocked.

- Document Findings & Recommendations — Produce a professional report summarising vulnerabilities and mitigations. Met by this report, including the findings register (Section 4), risk assessment (Section 5), and remediation plan (Section 6).

1.3 Scope & Legal Authorisation

All activities described in this report were performed exclusively within authorised, sandboxed lab environments provisioned for this internship task. No production systems, third-party accounts, or real customer data were accessed at any point.

Authorised environments used:

- Personal AWS Free Tier sandbox account — Account ID ending in 1111 — us-east-1 region
- CloudGoat scenario: iam_privesc_by_rollback — deployed in an isolated lab account with no production connectivity
- PwnedLabs: Assume Privileged Role with External ID — authorised, browser-hosted lab environment
- Cybr IAM Challenge Labs — authorised training environment

No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task. Claude was used only for analysis, explanation, and documentation support (see Section 2.3) — it was not used to execute any AWS CLI commands, generate the evidence files, or produce any of the JSON logs submitted as evidence.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1078	Valid Accounts	The escalation chain in Phase C used only legitimate, validly-issued temporary credentials at every hop — no exploit was required, only permissive trust relationships.
T1098	Account Manipulation	Chained AssumeRole calls effectively manipulate the acting identity's privilege level without modifying any user or role definition directly, evading controls that only monitor IAM entity changes.
T1550.001	Use of Alternate Authentication Material: Web Session Cookie	STS session tokens function analogously to session cookies — once issued, they grant the bearer the role's privileges for the token lifetime regardless of how they were obtained, which is why session duration limits were included in the Phase D hardening.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab environment consisted of a CloudGoat-deployed IAM privilege-escalation scenario inside an isolated AWS Free Tier sandbox account. No networking, storage, or compute resources beyond the IAM entities below were provisioned for this task.

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
cg-lab-user-solus	IAM User	Low-privilege starting identity with no directly attached policies	us-east-1 / created by CloudGoat
cg-lab-role-solus	IAM Role	Entry-point role with an overly permissive trust policy — hop 1 of the escalation chain	us-east-1 / trust policy hardened in Phase D
cg-lab-role-console_last_pass_role	IAM Role	Second-hop role carrying AdministratorAccess — final target of the escalation chain	us-east-1 / residual risk, not yet hardened
CloudGoat CLI	Local Python tool	Provisions and tears down the vulnerable scenario	Local workstation

Figure 1 — CloudGoat scenario role/trust relationship diagram (see phaseB_role_diagram evidence): cg-lab-user-solus → (unrestricted trust) → cg-lab-role-solus → (unrestricted trust) → cg-lab-role-console_last_pass_role (AdministratorAccess).

2.2 AWS CLI & Tool Configuration

Tool / Service	Version	Installation Method	Purpose in This Task
AWS CLI	v2.15.x	Pre-installed on lab workstation, verified with aws --version	Executing assume-role, get-role, and IAM query commands
CloudGoat	v2.x	git clone RhinoSecurityLabs/cloudgoat + pip install -r requirements.txt	Deploying the iam_privesc_by_rollback scenario
jq	1.6	sudo apt install jq -y	Parsing AssumeRolePolicyDocument JSON from CLI output
IAM Policy Simulator	Web Console	No install required	Validating the Phase D hardened policy before applying it
Claude	Web interface	No install required	Reviewing hardened policy JSON for completeness; explaining trust-policy condition semantics

Key Configuration Commands Used

aws configure # Default region name: us-east-1 / Default output format: json

aws sts get-caller-identity # Confirmed Account, UserId, and Arn of the lab user prior to any task execution

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
Claude	Explaining STS AssumeRole and trust-policy condition semantics; reviewing the Phase D	Claude did not execute any AWS CLI commands, did not generate the

AI Tool	Used For	What Was NOT Done with AI
	hardened policy JSON for completeness before applying it; assisting with report structure and wording	phaseC_assume_chain.json or phaseD_hardened_policy.json evidence content from a live AWS session, and did not produce any screenshots

All technical findings, CLI outputs, configurations, and evidence in this report reflect the intern's own lab execution.

3.0 Methodology — Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand how AWS STS issues temporary credentials and how attackers exploit weak trust relationships.
Estimated Time	~60 minutes
Evidence File(s)	phaseA_sts_output.png

Actions Taken

Reviewed AWS documentation on STS AssumeRole, focusing on how the Principal element and Condition block of a trust policy jointly determine who may assume a role.

Ran a basic role assumption to observe the returned credential structure:

aws sts assume-role --role-arn arn:aws:iam::<account-id>:role/ExampleRole --role-session-name testSession

Inspected the AccessKeyId, SecretAccessKey, and SessionToken returned, and confirmed their default one-hour expiry.

Discussed with reference to the task brief how these credentials could be abused if the trust policy's Principal or Condition block were too broad — e.g. a root-account principal with no ExternalId lets any authenticated identity in the account request the role.

Key Observations

STS issues short-lived credentials that are functionally equivalent to a static access key for their lifetime — the security of the entire mechanism therefore rests on the trust policy's Condition block, not on the temporary nature of the credentials alone.

Evidence Produced

- phaseA_sts_output.png — screenshot of a successful assume-role call showing the returned temporary credential structure

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Deploy the CloudGoat iam_privesc_by_rollback scenario to simulate an environment with misconfigured roles.
Estimated Time	~90 minutes
Evidence File(s)	phaseB_identity.png, phaseB_roles_table

Actions Taken

Initialised the CloudGoat scenario: ./cloudgoat.py create iam_privesc_by_rollback

Recorded the output credentials for the CloudGoat-created lab user (cg-lab-user-solus).

Verified connectivity with aws sts get-caller-identity and confirmed the Account, UserId, and Arn matched the newly created lab identity.

Documented every IAM role created by CloudGoat in a table, including cg-lab-role-solus and cg-lab-role-console_last_pass_role, along with their attached policies.

Key Observations

The environment deployed cleanly with no manual troubleshooting required. The baseline security posture was already weak by design — both roles created by CloudGoat had trust policies with no Condition block at all, which established the starting point against which the Phase D hardening was later measured.

Evidence Produced

- phaseB_identity.png — get-caller-identity output confirming the lab user's Account and Arn
- phaseB_roles_table — table of all IAM roles created by the scenario and their ARNs

Phase C: Core Execution

Phase Objective	Demonstrate how an attacker can exploit weak trust relationships to escalate privileges via AssumeRole chaining.
Estimated Time	~150 minutes
Evidence File(s)	phaseC_assume_chain.json

Actions Taken

Extracted the trust policy of the first target role: aws iam get-role --role-name cg-lab-role-solus | jq '.Role.AssumeRolePolicyDocument' — confirmed the Principal was the account root ARN with no Condition block.

Assumed cg-lab-role-solus using the starting lab user's credentials (hop 1) and captured the returned temporary credentials.

Using the hop-1 temporary credentials, inspected the trust policy of a second role, cg-lab-role-console_last_pass_role, and found it trusted the hop-1 role's ARN directly, again with no Condition block.

Chained a second assume-role call (hop 2) using the hop-1 credentials, successfully obtaining a session for cg-lab-role-console_last_pass_role.

Verified the resulting privilege level with aws iam list-attached-role-policies, confirming the final role carried the AWS-managed AdministratorAccess policy.

Recorded every command, output, and observation in the structured evidence log phaseC_assume_chain.json (see Deliverables, Section 8).

Key Observations

Both hops succeeded without any additional authentication factor, external ID, or source-IP restriction being required — the only barrier to full administrative access was the absence of any Condition block in either trust policy.

This matched the expectation set in Phase A: STS itself performed exactly as documented and enforced no additional scrutiny beyond what each role's trust policy specified. The risk lived entirely in the trust policy configuration, not in STS behaviour.

The two-hop chain is a realistic minimum case — in a larger account with more inter-role trust relationships, the reachable blast radius from a single low-privilege credential leak could be substantially larger.

Evidence Produced

- phaseC_assume_chain.json — full structured log of both AssumeRole hops, the intermediate trust-policy extractions, and the final privilege verification, with all secrets redacted

Phase D: Independent Extension & Advanced Implementation

Phase Objective	Apply security controls to prevent the abuse demonstrated in Phase C.
Estimated Time	~120 minutes
Evidence File(s)	phaseD_hardened_policy.json, phaseD_retest_log.txt

Actions Taken

Rewrote the AssumeRolePolicyDocument for cg-lab-role-solus, narrowing the Principal from the account root to the single legitimate IAM user ARN, and adding a Condition block requiring sts:ExternalId, aws:SourceIp within the approved lab CIDR, aws:MultiFactorAuthPresent, and a capped sts:DurationSeconds.

Validated the new policy's syntax using the IAM Policy Simulator before applying it, then used Claude to review the JSON for completeness against the four condition types specified in the task brief — the model flagged that a session-duration cap was a reasonable addition beyond the brief's minimum requirement, which was then included.

Re-ran the exact hop-1 command from Phase C without the new required parameters and confirmed it was now rejected with AccessDenied.

Re-ran the same call supplying the correct ExternalId and an MFA-authenticated session from the approved source IP, and confirmed it succeeded as expected.

Attempted the hop-2 chain again using the newly-issued hop-1 credentials against the still-unhardened cg-lab-role-console_last_pass_role trust policy, and confirmed it still succeeded — this was documented as a residual risk rather than treated as a hardening failure, since only the entry-point role was in scope for this pass.

Documented the full before-and-after comparison in phaseD_hardened_policy.json and the retest transcript in phaseD_retest_log.txt.

Key Observations

The hardening fully closed the specific escalation path demonstrated in Phase C at its entry point: the original Phase C hop-1 command now fails outright.

The extension work surfaced a finding beyond what Phase C alone showed: hardening a single role in a trust chain is necessary but not sufficient — every role along a potential chain needs equivalent conditions, or an attacker who reaches any unhardened intermediate role can still complete the chain. This is reflected as a distinct finding and risk in Sections 4 and 5.

Comparing the AI-assisted review to independent analysis, Claude's suggestion to cap sts:DurationSeconds was a genuinely useful addition not explicitly listed in the task brief's example condition block, and it aligns with NIST SP 800-53 AC-6 least-privilege guidance on limiting the exposure window of elevated sessions.

Evidence Produced

- phaseD_hardened_policy.json — before/after trust policy JSON with hardening controls documented
- phaseD_retest_log.txt — CLI transcript proving the original abuse path now fails, the hardened path succeeds under correct conditions, and the hop-2 residual risk

Risk Matrix (Phase D Output)

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood (1-5)	Impact (1-5)	Score	Rating
IAM Role: cg-lab-role-solus	Unrestricted trust allowing account-wide role assumption	T1078	4	4	16	Critical
IAM Role: cg-lab-role-console_last_pass_role	Privilege escalation via unhardened second-hop role chaining	T1098	3	5	15	Critical
STS Session Tokens (both roles)	No session-duration cap allowing long-lived elevated sessions	T1550.001	3	3	9	Medium

Score = Likelihood × Impact | 1-4: Low | 5-9: Medium | 10-14: High | 15-25: Critical

Phase E: Hosted Lab Integration

Phase Objective	Reinforce learning through external labs simulating real-world IAM exploitation.
Estimated Time	~180 minutes
Evidence File(s)	phaseE_lab1.png, phaseE_lab2.png

Actions Taken

Completed PwnedLabs — Assume Privileged Role with External ID, which required identifying a role whose trust policy's ExternalId condition could be inferred from an exposed configuration file, then using it to assume a privileged role.

Completed Cybr — IAM Challenge Escape Room, working through a sequence of IAM misconfiguration puzzles including an over-permissive trust policy scenario similar to the CloudGoat lab in Phase C.

Key Observations

The PwnedLabs scenario reinforced that ExternalId is only effective as a control when it is kept confidential — the lab's vulnerability was an ExternalId leaked via a misconfigured public file, not a weakness in the ExternalId mechanism itself. This nuance was not fully evident from the CloudGoat lab alone, since that scenario had no ExternalId in play at all before hardening.

Evidence Produced

- phaseE_lab1.png — PwnedLabs completion screenshot
- phaseE_lab2.png — Cybr IAM Challenge Escape Room completion screenshot

Phase F: Documentation & Reporting

Phase Objective	Compile findings into a professional security report suitable for management review.
Estimated Time	~120 minutes
Evidence File(s)	CSE_AWS_STS_Report.pdf

Actions Taken

Consolidated the screenshots, JSON logs, and observations from Phases A-E into the findings, risk assessment, and recommendations sections of this report.

Built the threat model and risk matrix (Sections 4 and 5) directly from the Phase C and Phase D evidence rather than generic guidance.

Cross-checked every recommendation in Section 6 against a specific finding number to ensure traceability.

Key Observations

Structuring the report around traceable findings — each with a MITRE reference, evidence file, and remediation — made it clear that the two Critical-rated risks (Findings 1 and 2) both stem from the same root cause: absent Condition blocks in trust policies, rather than two unrelated problems.

Evidence Produced

- CSE_AWS_STS_Report.pdf — this report, exported to PDF per the submission naming convention

4.0 Findings

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Unrestricted trust policy on entry-point role	Access Control Weakness	IAM Role: cg-lab-role-solus	T1078	Phase C	Critical	phaseC_assume_chain.json
2	Two-hop privilege escalation to AdministratorAccess	Privilege Escalation Path	IAM Role: cg-lab-role-console_last_pass_role	T1098	Phase C	Critical	phaseC_assume_chain.json
3	Missing ExternalId enables confused-deputy risk in cross-account style trust	IAM Policy Gap	IAM Roles (both)	T1078	Phase C / D	High	phaseD_hardened_policy.json
4	No session-duration limit on issued STS credentials	IAM Policy Gap	STS (both roles)	T1550.001	Phase D	Medium	phaseD_hardened_policy.json
5	Hop-2 role remains unhardened after entry-point remediation	Privilege Escalation Path (residual)	IAM Role: cg-lab-role-console_last_pass_role	T1098	Phase D	High	phaseD_retest_log.txt

4.2 Detailed Finding Analysis

Finding 1: Unrestricted trust policy on entry-point role

Finding Title	Unrestricted trust policy on entry-point role
Type / Category	Access Control Weakness
Cloud Service / Resource	IAM Role: cg-lab-role-solus
Description	The AssumeRolePolicyDocument for cg-lab-role-solus scoped its Principal to the account root ARN (arn:aws:iam::111111111111:root) with no Condition element, meaning any authenticated IAM principal in the account could call sts:AssumeRole successfully with no additional verification.
How Identified	aws iam get-role --role-name cg-lab-role-solus | jq '.Role.AssumeRolePolicyDocument', followed by a live assume-role call that succeeded without any External ID, MFA, or source-IP requirement.
MITRE ATT&CK Reference	T1078 — Valid Accounts
Business / Security Impact	An attacker who compromises any credential in the account — even one with no directly attached IAM policy — can immediately pivot into this role's permission set with no additional exploitation required.
Severity / Impact	Critical
Evidence Reference	phaseC_assume_chain.json, steps 2-3

Finding 2: Two-hop privilege escalation to AdministratorAccess

Finding Title	Two-hop privilege escalation to AdministratorAccess
Type / Category	Privilege Escalation Path
Cloud Service / Resource	IAM Role: cg-lab-role-console_last_pass_role
Description	Using hop-1 temporary credentials from Finding 1, a second AssumeRole call to cg-lab-role-console_last_pass_role succeeded, granting a session with the AWS-managed AdministratorAccess policy attached — full administrative control of the sandbox account.
How Identified	aws iam list-attached-role-policies --role-name cg-lab-role-console_last_pass_role, run using the hop-1 session, confirmed AdministratorAccess.
MITRE ATT&CK Reference	T1098 — Account Manipulation
Business / Security Impact	The combination with Finding 1 means the entire escalation from a zero-privilege identity to full account control required only two CLI commands and no exploit code, representing a Critical business risk in any account with real workloads.
Severity / Impact	Critical
Evidence Reference	phaseC_assume_chain.json, steps 4-6

Finding 3: Missing ExternalId enables confused-deputy risk

Finding Title	Missing ExternalId enables confused-deputy risk

Type / Category	IAM Policy Gap
Cloud Service / Resource	IAM Roles: cg-lab-role-solus, cg-lab-role-console_last_pass_role
Description	Neither role's trust policy included an sts:ExternalId condition. While both roles are same-account in this lab, the same trust pattern used for cross-account access without an ExternalId is a documented AWS anti-pattern that enables confused-deputy attacks, where a third party is tricked into assuming a role on an attacker's behalf.
How Identified	Manual review of both AssumeRolePolicyDocument statements against the AWS cross-account access best-practice guidance referenced in Phase A.
MITRE ATT&CK Reference	T1078 — Valid Accounts
Business / Security Impact	If this trust pattern were reused for genuine cross-account access, any party who learns the role ARN could potentially trigger unauthorised assumption without needing to know any account-specific secret.
Severity / Impact	High
Evidence Reference	phaseD_hardened_policy.json (before/after comparison)

Finding 4: No session-duration limit on issued STS credentials

Finding Title	No session-duration limit on issued STS credentials
Type / Category	IAM Policy Gap
Cloud Service / Resource	STS temporary credentials for both roles
Description	Prior to hardening, neither role's trust policy constrained sts:DurationSeconds, meaning any successfully assumed session could request up to the role's maximum session duration, extending the exposure window of any leaked temporary credential.
How Identified	Comparison of pre- and post-hardening AssumeRolePolicyDocument statements during Phase D.
MITRE ATT&CK Reference	T1550.001 — Use of Alternate Authentication Material: Web Session Cookie
Business / Security Impact	A longer-lived elevated session increases the window in which a leaked temporary credential remains useful to an attacker, directly increasing the impact of any credential-theft scenario.
Severity / Impact	Medium
Evidence Reference	phaseD_hardened_policy.json

Finding 5: Hop-2 role remains unhardened after entry-point remediation

Finding Title	Hop-2 role remains unhardened after entry-point remediation
Type / Category	Privilege Escalation Path (residual)
Cloud Service / Resource	IAM Role: cg-lab-role-console_last_pass_role
Description	After hardening cg-lab-role-solus in Phase D, the retest confirmed that a hop-1 session obtained through the hardened, MFA-gated path could still successfully chain into cg-lab-role-console_last_pass_role, because that role's trust policy was

	not part of this hardening pass and still trusts the hop-1 role ARN unconditionally.
How Identified	Retest of the Phase C hop-2 command using freshly issued, hardened hop-1 credentials (phaseD_retest_log.txt, Test 3).
MITRE ATT&CK Reference	T1098 — Account Manipulation
Business / Security Impact	Illustrates that partial remediation of a trust chain provides a false sense of security — the account-wide blast radius is unchanged until every role in the chain carries equivalent conditions.
Severity / Impact	High
Evidence Reference	phaseD_retest_log.txt, Test 3

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Finding 1 & 2 — Unrestricted trust / chained escalation	AssumeRole events in CloudTrail from an unexpected source identity or in rapid succession across roles	IAM Access Analyzer external-access findings; Condition-based trust policies	AWS IAM Access Analyzer; AWS Config rule iam-role-managed-policy-check; CloudWatch metric filter on AssumeRole events
Finding 3 — Missing ExternalId	AssumeRole call succeeding with no sts:externalId parameter present in the CloudTrail event	Mandate sts:ExternalId on every cross-account-style trust relationship	AWS Config custom rule / Security Hub custom insight scanning trust policies for missing Condition blocks
Finding 4 — No session-duration cap	Long DurationSeconds values requested in AssumeRole CloudTrail events	Cap sts:DurationSeconds via trust-policy Condition and role MaxSessionDuration setting	IAM role MaxSessionDuration attribute; CloudWatch alarm on high-duration AssumeRole requests
Finding 5 — Residual chained risk	Successful AssumeRole into cg-lab-role-console_last_pass_role sourced from another role's assumed-role ARN	Apply identical Condition hardening to every role in a trust chain, not only the entry point	AWS IAM Access Analyzer to map the full trust graph before declaring remediation complete

5.2 Risk Exposure Over Time

Risk 1 — Full account takeover via unhardened trust chains:

As shown by Findings 1, 2, and 5, an attacker who obtains any low-privilege credential in this account can currently reach AdministratorAccess in two hops. If left unaddressed, this risk compounds over time as more roles and services are added — each new role with a similarly loose trust policy becomes another potential hop, expanding the attack surface without any corresponding increase in monitoring.

Risk 2 — Partial remediation creating false confidence:

Finding 5 demonstrates that hardening only the entry-point role (cg-lab-role-solus) does not close the overall escalation path. If this report's Phase D fix were treated as "complete" without extending the same conditions to cg-lab-role-console_last_pass_role, stakeholders would have a false sense that the risk was resolved, while the same AdministratorAccess exposure remains reachable via the unhardened second hop.

Risk 3 — Long-lived sessions extending breach impact:

Finding 4's absence of a session-duration cap means that even after a credential-theft incident is detected and the underlying access key rotated, any already-issued STS session for the affected role remains valid for up to its full duration. Over time, as more roles are created without a MaxSessionDuration policy, incident responders lose the ability to guarantee that revoking a credential fully terminates an attacker's access within a predictable window.

6.0 Recommendations

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Finding 1 — Unrestricted trust on entry-point role	Apply the Phase D hardened policy in production: scope Principal to the exact ARN, add sts:ExternalId, aws:SourceIp, and aws:MultiFactorAuthPresent conditions, e.g. aws iam update-assume-role-policy --role-name cg-lab-role-solus --policy-document file://phaseD_hardened_policy.json	Immediate	NIST SP 800-53 AC-6 / CIS Control 6
2	Finding 2 & 5 — Chained escalation to AdministratorAccess	Extend identical Condition hardening to cg-lab-role-console_last_pass_role's trust policy, and run AWS IAM Access Analyzer to confirm no further unhardened hops exist in the trust graph	Immediate	MITRE T1098 / OWASP Cloud Top 10 IAM-01
3	Finding 3 — Missing ExternalId	Add sts:ExternalId to every trust policy used for cross-account-style access patterns, and store the ExternalId as a secret rather than in any publicly accessible configuration file (per the Phase E PwnedLabs lesson)	Short-term	AWS cross-account access best practice / NIST SP 800-53 AC-2
4	Finding 4 — No session-duration cap	Set MaxSessionDuration to 3600 seconds on both roles and mirror the limit in the trust policy's sts:DurationSeconds condition	Short-term	NIST SP 800-53 AC-6

6.2 Strategic & Operational Improvements

- Run AWS IAM Access Analyzer across the account on a scheduled basis to automatically surface any role whose trust policy grants access without an ExternalId, source-IP, or MFA condition — directly addressing Findings 1, 2, 3, and 5.
- Implement Service Control Policies (SCPs) at the AWS Organization level that deny iam:UpdateAssumeRolePolicy actions which would remove an existing Condition block, preventing hardened trust policies from silently regressing.
- Configure a CloudWatch metric filter and alarm on chained AssumeRole events (an AssumeRole call whose calling identity ARN itself contains "assumed-role"), giving near-real-time visibility into the exact escalation pattern demonstrated in Phase C.
- Adopt a standard MaxSessionDuration ceiling (e.g. 3600 seconds) as an account-wide IAM guardrail for all newly created roles, rather than relying on each role author to set it individually.
- Require every new cross-account or service-to-service trust policy to pass through IAM Policy Simulator validation and a peer review checklist before deployment, so gaps like the ones found here are caught before a role reaches production.

7.0 Reflection

Q1 How could an attacker leverage chained AssumeRole operations to escalate privileges across accounts?

As demonstrated in Phase C, an attacker only needs one valid low-privilege credential and read access to enumerate IAM roles. From there, they can extract each role's trust policy, identify any that trust their current identity (or the account root) without a Condition block, and assume it. Each successful assumption yields a new identity that may itself be trusted by a further role, so the attacker repeats the process — exactly the two-hop chain from cg-lab-user-solus to AdministratorAccess. Across accounts, the same pattern applies when a role in Account B trusts a role ARN in Account A without an ExternalId: compromising Account A becomes sufficient to pivot into Account B.

Q2 What business risks arise from overly permissive cross-account trusts?

The core risk is loss of the security boundary that separate AWS accounts are meant to provide. As Finding 2 shows, an overly permissive trust relationship can turn a minor compromise (a single low-privilege credential) into full administrative control, which in a production context could mean data exfiltration, resource destruction, or the ability to disable logging and alerting entirely. Beyond direct technical impact, this also creates compliance exposure — ISO 27001 A.9.2.3 specifically requires managed privileged access, and an unrestricted trust policy like the one in Finding 1 would fail that control on audit.

Q3 How do external IDs mitigate third-party access risks?

An ExternalId acts as a shared secret that the trusting account requires in addition to the correct Principal before honouring an AssumeRole request, which specifically defeats the confused-deputy scenario where a legitimate third party (for example, a SaaS vendor with a standing cross-account role) is tricked by another customer into assuming a role on the attacker's behalf. The Phase E PwnedLabs exercise reinforced that this only works if the ExternalId is kept confidential — that lab's vulnerability was an ExternalId leaked in an exposed configuration file, not a flaw in the mechanism itself, which is why Finding 3's remediation explicitly calls for secret storage, not just the presence of the condition.

Q4 If you were designing a multi-account architecture for a financial institution, what additional controls would you implement?

Beyond the ExternalId, source-IP, and MFA conditions applied in Phase D, I would enforce a short MaxSessionDuration across every role (as recommended for Finding 4), require SCPs that block any role from removing Condition blocks from its own trust policy, and use IAM Access Analyzer continuously rather than as a one-time check, since Finding 5 showed how easily a partially remediated trust chain can look secure while still containing a viable escalation path. For a financial institution specifically, I would also add hardware-backed MFA requirements for any role capable of reaching production financial-data resources, and route every cross-account AssumeRole event through a dedicated CloudWatch alarm reviewed by the security team, not just general CloudTrail retention.

8.0 Deliverables Submission Checklist

Phase	Deliverable Description	Your Filename	Status
Phase A	Theory evidence — assume-role output screenshot	phaseA_sts_output.png	Complete
Phase B	Environment setup evidence — identity verification screenshot	phaseB_identity.png	Complete
Phase C	Core execution evidence — structured JSON log of the assume-role chain	phaseC_assume_chain.json	Complete
Phase D	Extension evidence — hardened trust policy JSON and retest transcript	phaseD_hardened_policy.json, phaseD_retest_log.txt	Complete
Phase D	Hosted lab completion — PwnedLabs / Cybr proof	phaseE_lab1.png, phaseE_lab2.png	pending
Report	This report — exported as CSE_[TaskID]_[Initials]_[YYYYMMDD].pdf	CSE_AWS-L-027-04_SK.20260831.pdf	Complete

Submission naming convention: Report PDF — CSE_[TaskID]_[YourInitials]_[YYYYMMDD].pdf; Evidence folder — [TaskID]_Evidence_[YourInitials]/ zipping all phase files together.

9.0 Additional Observations

- The CloudGoat teardown command (./cloudgoat.py destroy iam_privesc_by_rollback) should be run promptly after evidence capture to avoid any lingering Free Tier IAM resource charges, though IAM roles themselves carry no direct cost.
- This task connects directly to the programme's S3 Access Control and Logging & Monitoring tracks: the same CloudTrail-based detection approach recommended in Section 5.1 for AssumeRole chaining is reusable for detecting anomalous S3 access patterns in other tasks.
- Feedback for the programme team: the task brief's Phase D example condition block (ExternalId + SourceIp only) is a good minimum, but explicitly prompting interns to also consider MFA and session-duration conditions — as this report did — would make the hardening exercise more representative of AWS's own cross-account access best-practice guidance.

Declaration & Sign-Off

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	[redacted]
Signature	[redacted]
Date of Submission	31 Aug 2026

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0